LJ University

Department of Computer Applications

ARSH PERFUME

E-Commerce Website

PROJECT DOCUMENTATION

MASTER OF COMPUTER APPLICATION (INTEGRATED)

GROUP MEMBERS	Khizar Gausee Khuzaim Karimi Ayan Mandli
SUBMITTED TO	Nirav Rajgor sir
ACADEMIC YEAR	2026-2027
SUBMISSION DATE	28-10-2026
DIV-ROLL NO.	D14 — Khizar Gausee D24 — Khuzaim Karimi D12 — Ayan Mandli

INTERNAL GUIDE

Nirav Rajgor sir

TABLE OF CONTENTS

1. INTRODUCTION 4

1.1 Existing System 4

1.2 Need for the New System 5

1.3 Objective of the New System 5

1.4 Problem Definition 6

1.5 Core Components 6

1.6 Project Profile 7

1.7 Assumptions and Constraints 7

1.8 Advantages and Limitations 8

2. PROJECT PLANNING 9

2.1 Team Details and Roles 9

2.2 Proposed Timeline (Gantt Chart) 10

3. REQUIREMENT DETERMINATION & ANALYSIS 11

3.1 Requirement Determination 11

3.2 Targeted Users 13

4. SYSTEM DESIGN 14

4.1 Use Case Diagram 14

4.2 Class Diagram 15

4.3 Interaction Diagram 16

4.4 Activity Diagram 17

4.5 Data Dictionary 18

TABLE OF CONTENTS (continued)

5. DEVELOPMENT 38

5.1 Development Approach 38

5.2 Coding Standards 39

5.3 GitHub Repository Details 40

5.4 AI Tools / Libraries / APIs Used 41

5.5 Screenshots & Implementation Details 42

5.6 Installation and Configuration 46

5.7 Testing and Validation 47

5.8 Deployment and Maintenance 48

6. PROPOSED ENHANCEMENTS 49

7. CONCLUSION 50

8. BIBLIOGRAPHY 51

> The eight chapters follow the supplied UDP documentation structure. Sections 5.6–5.8 add setup, validation and operating information. The data dictionary covers all 36 migration-declared tables.

1. INTRODUCTION

ARSH PERFUME is a web-based e-commerce application for presenting and selling attars, roll-on perfumes and fragrance oils. The application provides a branded customer storefront and a protected administration panel within one Laravel project. It supports product discovery, variant-level pricing and stock, shopping carts, checkout, payments, order fulfilment and customer account management.

The website is designed for Indian delivery addresses and INR transactions. Customers can browse the catalogue without an account, while registered customers gain a wishlist, saved addresses and order history. The administrator manages the catalogue, stock, orders, content and business settings from a centralized workspace.

1.1 Existing System

The comparison baseline is a conventional small-store workflow in which product information is shared over messages or a basic catalogue, orders are recorded in spreadsheets, and payment and dispatch updates are checked manually. This baseline explains the problem addressed by the application; it is not a claim that a formal field survey of the store was conducted.

1. Customer asks for available fragrances, sizes and prices.
2. Store staff check stock and communicate availability.
3. Order and address details are recorded manually.
4. Payment is checked separately and the parcel is dispatched.
5. Stock sheets and customer updates are maintained independently.

Limitations of the comparison workflow

- Repeated manual communication for product information and order status.
- Separate price, stock and order records can become inconsistent.
- Historical sales and customer information are difficult to consolidate.
- Payment references and cancellations require careful reconciliation.

Source: routes/web.php; routes/admin.php; app/Services/OrderService.php.

1.2 Need for the New System

A single online platform is needed to present accurate product information, preserve purchase records and coordinate the relationship between customer orders, payments and stock. A customer should be able to discover a fragrance, choose an available size and place an order without depending on repeated manual messages.

For the store owner, the system should provide one place to maintain prices, stock, promotional content, order status and reporting. Payment states and fulfilment states must remain distinct so that a customer-entered transaction reference or a browser redirect cannot be mistaken for confirmed receipt of money.

1.3 Objective of the New System

- Provide a searchable, responsive catalogue with categories, fragrance filters and product details.
- Maintain price, sale price, availability and low-stock thresholds for each product variant.
- Support guest and signed-in carts, coupons and validated checkout.
- Calculate totals on the server using shared shipping, discount and tax rules.
- Support COD, verified Razorpay payments and direct UPI with manual bank-receipt verification.
- Reserve stock during order placement and restore cancelled reservations once.
- Allow customers to manage their accounts, addresses, wishlist and own orders.
- Provide an administrator workspace for products, inventory, customers, content and reports.
- Maintain order history, stock movements and an administrator activity trail.
- Document configuration, testing, known limitations and operation for future maintenance.

Source: app/Services/; app/Models/; config/arsh.php.

1.4 Problem Definition

Develop a maintainable e-commerce website for ARSH PERFUME that connects product browsing, variant selection, cart management, validated checkout, payment handling, stock reservation and order fulfilment. The application must prevent unauthorized access to customer records, avoid trusting browser-submitted totals and preserve the purchased item and address details as historical snapshots.

The scope includes a public storefront, customer account area and administrator panel. The source does not implement a marketplace for multiple sellers, a machine-learning recommendation service or a separate mobile application.

1.5 Core Components

Module	Responsibility
User management	Email/password accounts, password reset, active-account checks and optional Google sign-in.
Catalogue	Categories, product descriptions, images, fragrance details and purchasable sizes.
Cart and checkout	Guest/customer carts, coupon validation, Indian addresses and order placement.
Payment management	COD, signed Razorpay callbacks/webhooks and manually verified direct UPI.
Inventory	Variant stock, reservation, adjustments and movement history.
Order management	Status transitions, tracking, invoices, cancellations, returns and refunds.
Customer engagement	Wishlist, verified-purchase reviews, contact enquiries, FAQs and newsletter.
Administration and reporting	Dashboard, business settings, CMS, reports, CSV exports and activity logs.

Source: routes/; app/Http/Controllers/; app/Services/.

1.6 Project Profile

Attribute	Details
Project name	ARSH PERFUME — E-Commerce Website
Project type	Web-based retail e-commerce application
Academic programme	Master of Computer Application (Integrated)
Academic year	2026-2027
Team size	3 students
Internal guide / submitted to	Nirav Rajgor sir
Backend	PHP ^8.2, Laravel ^12.0 and Eloquent ORM
Frontend	Blade, Tailwind CSS 4, Alpine.js 3 and Vite 7
Database	MySQL-oriented schema; SQLite used by automated tests
Payments	Razorpay, COD and direct UPI QR
Users	Visitors/guests, registered customers and administrators
Development environment	Windows / XAMPP project workspace
Submission date	28-10-2026

1.7 Assumptions and Constraints

Assumptions

- The owner supplies accurate product claims, images, prices, stock and business policies.
- Customers provide valid contact and delivery information.
- Administrators verify UPI receipts against the actual receiving account.
- The deployed server has a working database, storage, mail configuration and scheduled background processes.

Constraints

Checkout uses INR and configured Indian states/territories with six-digit PIN validation. Online integrations depend on provider credentials and connectivity. Direct UPI requires administrator review. Catalogue and policy seeds contain intentionally incomplete/unpublished content.

Source: composer.json; package.json; app/Http/Requests/CheckoutRequest.php; database/seeders/.

1.8 Advantages and Limitations

Advantages

- A centralized product catalogue and administration workspace reduce repeated data entry.
- Variant-level pricing and stock support different fragrance sizes.
- Server-side total calculation keeps cart, checkout and order amounts consistent.
- Transactional placement and row locking protect stock during competing checkouts.
- Item, address and receiving-account snapshots preserve purchase-time information.
- Signed payment verification and duplicate-event handling support payment integrity.
- Role checks and ownership policies protect administration and customer records.
- Reports, history and movement logs support operational review.
- Responsive layouts support desktop and mobile browser use.

Current limitations

- Direct UPI and UPI refunds involve external bank verification or transfer.
- CSV product export works; the product-import route lacks its controller implementation.
- Category ordering uses numeric sort values rather than drag-and-drop interaction.
- SMS/WhatsApp notifications, multiple currencies and multilingual content are not implemented.
- The automated suite currently contains two existing failures, described in Section 5.7.
- A live gateway transaction and a real Google consent round trip require configured credentials and separate verification.
- No measured production-load or uptime guarantee is established by this report.

> This report describes the implemented source and observed checks. Proposed enhancements and planning allocations are clearly distinguished from verified functionality.

2. PROJECT PLANNING

2.1 Team Details and Roles

Name of the Student	Div / Roll No.	Proposed Role of the Student
Khizar Gausee	D14	Frontend implementation, responsive screens and customer usability.
Khuzaim Karimi	D24	Backend services, authentication, checkout and payment integration.
Ayan Mandli	D12	Database/inventory design, testing and project documentation.

> The member names and roll entries are supplied by the project group. The responsibility split is a proposed planning allocation for this report; it is not presented as a verified contribution history.

Coordination and review plan

- Review requirements against the implemented routes, services and database migrations.
- Agree on interfaces between catalogue, cart, pricing, orders, payments and inventory.
- Keep customer-facing and administrator changes reviewable and consistent.
- Review access checks, stock effects and payment outcomes before accepting a change.
- Maintain screenshots and documentation when screens or configuration change.

Expected deliverables

- Working project source and database migrations.
- Public storefront, customer account flows and administrator modules.
- Local setup, environment configuration and operating instructions.
- Automated test evidence with unresolved issues recorded.
- Academic report, diagrams, data dictionary and implementation screenshots.

The next page gives the proposed twelve-week schedule requested by the reference format. It is an academic planning model ending on the supplied submission date, not an assertion of actual work dates.

2.2 Proposed Timeline (Gantt Chart)

Illustrative twelve-week plan: 06-08-2026 to 28-10-2026. Week numbers represent planned work windows; overlapping activities permit integration and documentation alongside implementation.

Activity	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12
Requirement analysis	●	●										
System design		●	●									
Database design			●	●								
UI / responsive frontend			●	●	●	●						
Backend / authentication				●	●	●	●					
Payments / inventory						●	●	●	●			
Integration and testing								●	●	●	●	
Documentation						●	●	●	●	●	●	●
Final review / presentation											●	●

Planned milestones

- Week 2: requirements baseline and user/scope review.
- Week 4: system/database design baseline.
- Weeks 7–9: core workflows and payment/inventory integration.
- Weeks 11–12: verification, report review and submission on 28-10-2026.

Risk and dependency planning

Payment-provider activation, Google credentials, owner-supplied catalogue content and server configuration can affect readiness. Keep local simulation and mock-based tests available, while recording which real integrations remain unverified. Reserve time for test failures, publication review and documentation correction.

3. REQUIREMENT DETERMINATION & ANALYSIS

3.1 Requirement Determination

The requirements below are derived from the repository, validators, routes, service rules, migration definitions and existing test cases. No interviews, customer surveys or benchmark measurements are claimed. Requirements describe observable behavior and the checks needed to accept it.

ID	Functional requirement	Acceptance condition
FR01	Browse and search the catalogue	Active catalogue pages support keyword, category, family, price and stock filters.
FR02	Manage product variants	Each size carries a unique SKU, price, active state and stock.
FR03	Authenticate users	Email/password and configured Google sign-in preserve safe customer roles.
FR04	Maintain cart contents	Guest/customer carts validate quantities and merge on sign-in.
FR05	Apply coupons	Dates, eligibility, minimums, usage limits and discount caps are checked.
FR06	Place orders safely	Checkout rechecks price/stock in a transaction and snapshots purchase data.
FR07	Handle payments	COD, verified Razorpay and manually verified UPI follow their respective rules.
FR08	Track and cancel orders	Customers see their permitted orders; cancellation respects status rules.
FR09	Control inventory	Stock changes create movement records and do not become negative.
FR10	Administer the store	Active administrators manage catalogue, content, orders and settings.
FR11	Support engagement	Wishlist, delivered-purchase reviews, contact and newsletter flows exist.
FR12	Report and audit	Dashboard/reports, order histories and admin actions support review.

Source: routes/; app/Http/Requests/; app/Services/; tests/Feature/.

3.1 Requirement Determination (continued)

Non-Functional Requirements

Area	Requirement / design expectation	Evidence or practical limit
Security	Protect roles, ownership, sessions, input and payment verification.	Middleware, policies, request validation, signatures and security tests.
Data integrity	Prevent duplicate orders/events and inconsistent stock/totals.	Idempotency keys, transactions, row locks and movement records.
Usability	Provide clear navigation, validation and responsive screens.	Blade layouts, customer/admin views and saved preview screenshots.
Maintainability	Separate request, business and persistence responsibilities.	Controllers, Form Requests, services, models and reusable helpers.
Performance	Use efficient queries, built assets and cached configuration/settings.	Implementation mechanisms exist; no load-tested response-time target is claimed.
Reliability	Retain purchase history and run queued/scheduled work consistently.	Snapshots, histories, queue worker and scheduler; deployment operation is required.
Recoverability	Back up the database/media and rehearse restoration.	Operating requirement; this report does not claim a completed restore rehearsal.
Compatibility	Support current desktop/mobile browser workflows.	Responsive implementation; broad browser-matrix testing remains a deployment task.

Hardware and software requirements

Development requires a machine capable of running PHP, MySQL, Composer, Node/npm and a modern browser. The project workspace uses Windows/XAMPP. No formal minimum RAM/CPU specification is asserted without measurement. Use PHP 8.2+, MySQL 8 and Node compatible with Vite 7 (20.19+ or 22.12+), with appropriate PHP database, file, XML and image extensions.

Source: composer.json; package.json; phpunit.xml; config/.

3.2 Targeted Users

User Type	Role	Responsibilities / permissions
Visitor / guest	Prospective buyer	Browse, search, manage a session cart, use enabled guest checkout, track by order number/email and contact the store.
Registered customer	Account holder	Manage profile/password, addresses, wishlist and own order history; cancel permitted orders and review delivered purchases.
Administrator	Store operator	Manage catalogue, images, inventory, orders, payment confirmation, content, customers, settings and reports.
Payment provider	External service actor	Provide Razorpay checkout/payment records and signed event notifications.
Google identity provider	Optional external service actor	Return a verified identity through the configured OAuth flow.

Role boundaries

Only active administrators may enter /admin. Public registration cannot grant an administrator role. Customers cannot access another customer's orders or addresses. A guest order is accessible through its permitted placing-session context; tracking uses the separate order-number/email lookup.

User journey

1. Discover products through home, shop or collections.
2. Choose an available variant and build the cart.
3. Review coupon, shipping and tax calculations.
4. Provide delivery information and choose a payment method.
5. Complete payment instructions and retain the order number.
6. Track fulfilment; administrators process verified orders.

Source: bootstrap/app.php; app/Support/OrderAccess.php; app/Policies/; routes/auth.php.

4. SYSTEM DESIGN

4.1 Use Case Diagram

The system boundary separates public shopping and customer-account actions from administrator operations. Razorpay is an external payment actor. Google OAuth is an optional identity integration described in the development chapter.

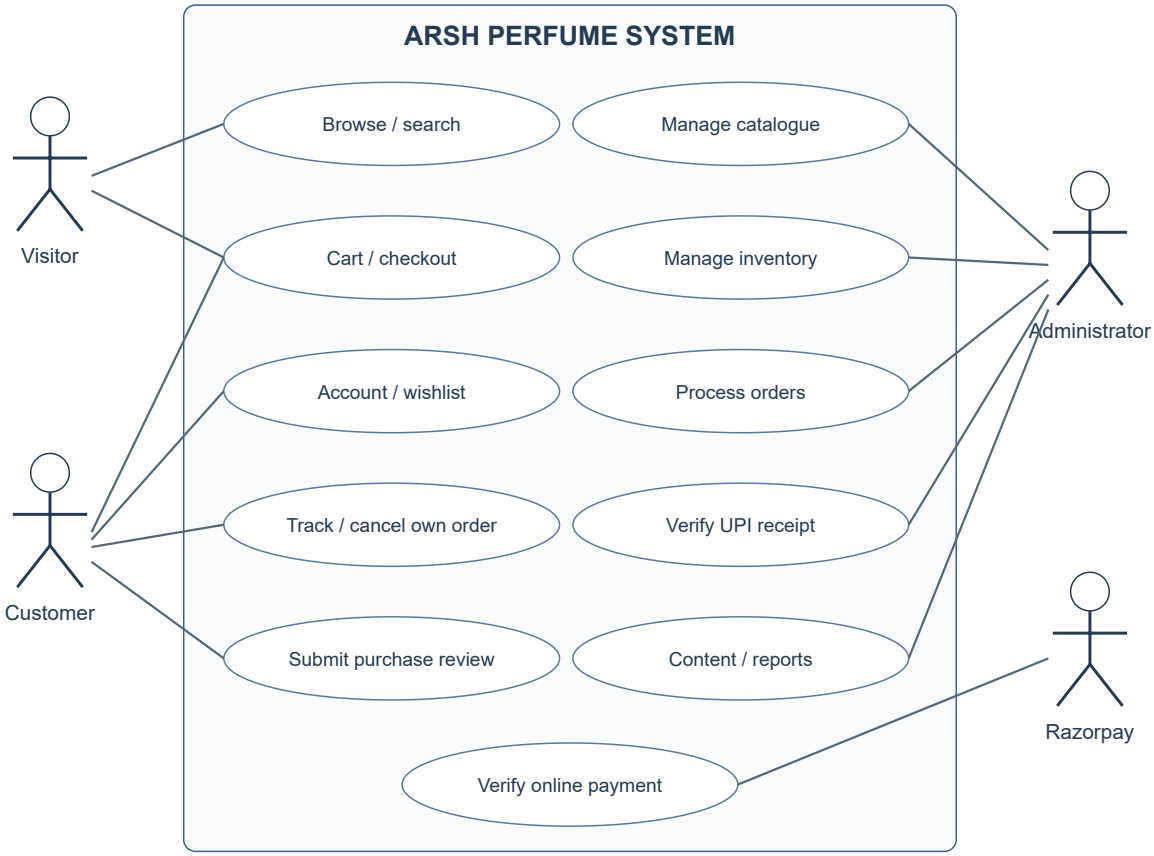


Figure 4.1: Main actors and use cases of ARSH PERFUME.

> Order ownership, customer cancellation states, review eligibility and administrator payment verification apply to the associated use cases.

4.2 Class Diagram

The diagram shows central Eloquent classes and persisted attributes. Multiplicity 1–* denotes a one-to-many association; 0..1 denotes an optional association. Order items retain snapshots, so later product edits do not rewrite the original sale.

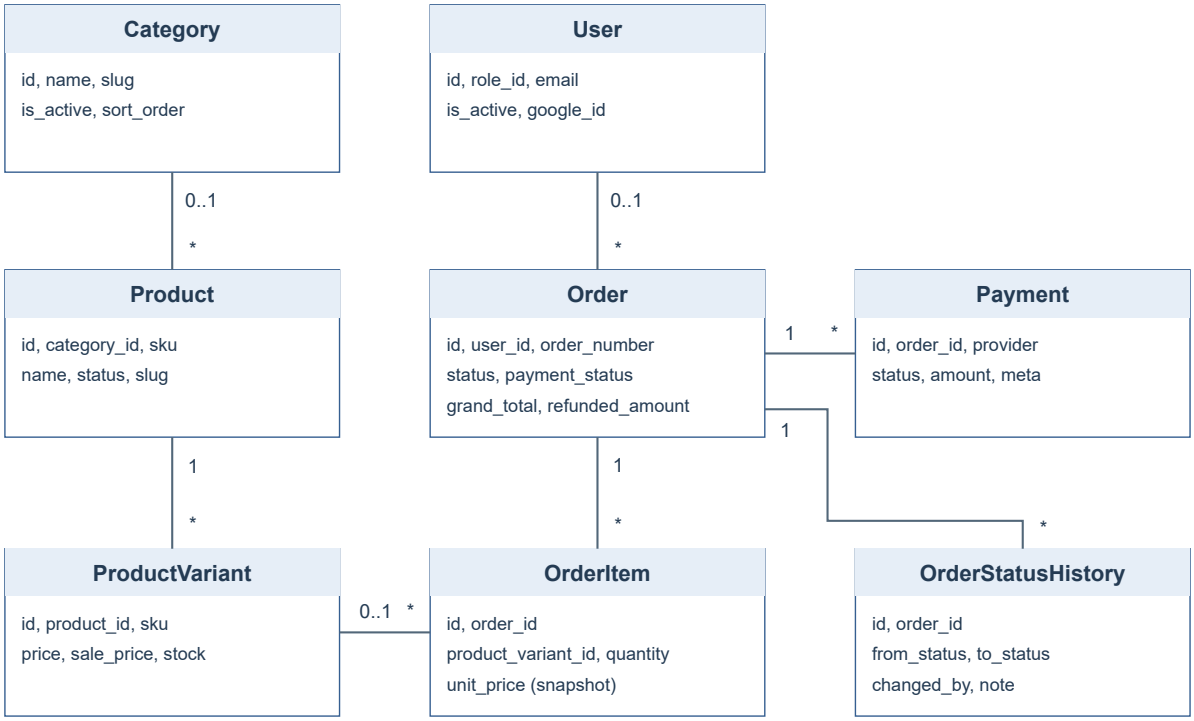


Figure 4.2: Core class relationships and cardinalities.

Additional domain classes

Role, UserAddress, Cart/CartItem, Wishlist/WishlistItem, Coupon/CouponUsage, InventoryMovement, Review, Page, Banner, Faq and Setting support the remaining modules. Price and stock belong to ProductVariant. Guest orders have a nullable user_id. The full table definitions follow in Section 4.5.

Source: app/Models/; database/migrations/.

4.3 Interaction Diagram

This sequence summarizes checkout interaction. The OrderService transaction validates and records the order before method-specific payment handling. Razorpay confirmation uses verified signatures and amounts; direct UPI requires actual receipt verification by an administrator.

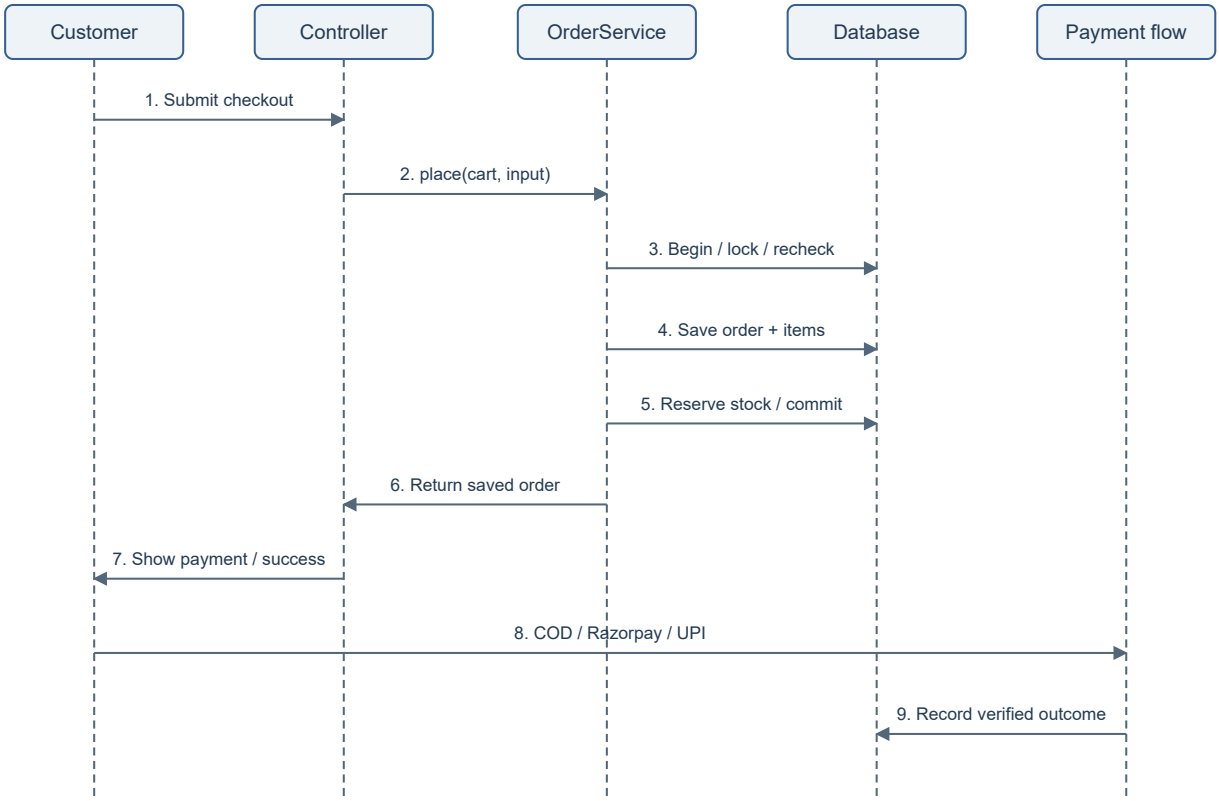


Figure 4.3: Checkout interaction / sequence diagram.

> A repeated idempotency key returns the existing order. A browser redirect or submitted UPI reference does not independently establish that payment was received.

4.4 Activity Diagram

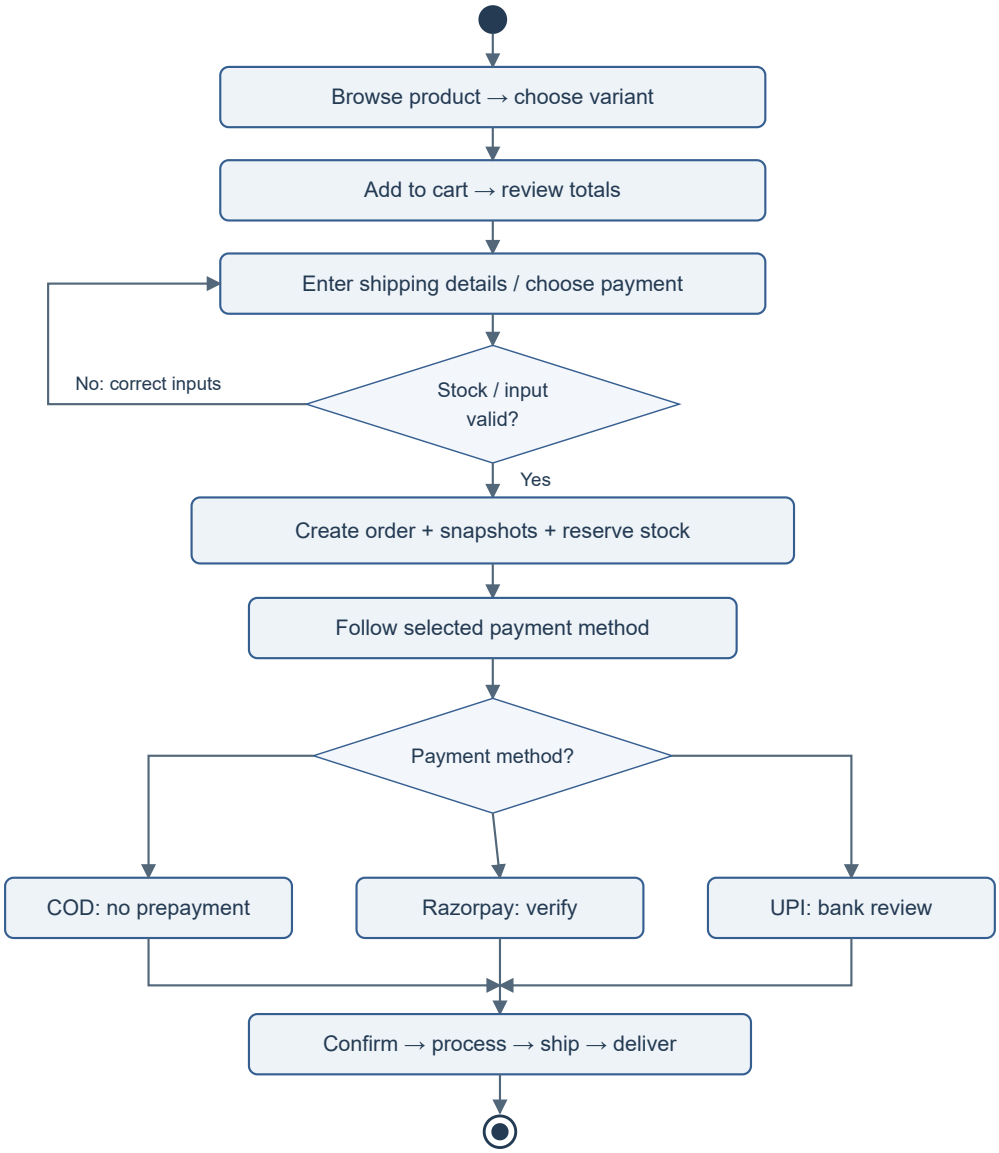


Figure 4.4: Main shopping and fulfilment activity flow.

Unpaid online orders cannot advance through fulfilment. COD collection can be recorded explicitly and is automatically marked on delivery. Cancellation follows allowed state transitions and restores a stock reservation once.

4.5 Data Dictionary

The migration source declares 36 tables: 29 application tables and 7 Laravel support tables. Laravel also creates its migrations bookkeeping table. The following dictionary reproduces field declarations, constraints, defaults, indexes and relationships from the maintained migration-derived reference; it contains no customer records or credentials.

Domain	Tables
Accounts	roles, users, user_addresses
Catalogue	categories, products, product_images, product_variants, reviews
Shopping	carts, cart_items, wishlists, wishlist_items
Coupons	coupons, coupon_products, coupon_categories, coupon_usages
Fulfilment	orders, order_items, order_status_histories, inventory_movements
Payments	payments, payment_webhook_events
Content / operations	contact_enquiries, newsletter_subscribers, banners, pages, faqs, settings, admin_activity_logs
Framework support	password_reset_tokens, sessions, cache, cache_locks, jobs, job_batches, failed_jobs

Reading the schema declarations

- `id()`: auto-incrementing primary key. `foreignId()`: related record identifier.
- `nullable()`: optional value. `unique()`: uniqueness constraint.
- `decimal(name, precision, scale)`: fixed-precision decimal value.
- `timestamps()`: created_at and updated_at. `softDeletes()`: deleted_at.
- `constrained()` and deletion rules identify foreign-key behavior.

Money is stored as decimal database values and calculated in integer paise for checkout. Order item/address and UPI receiving-account snapshots preserve purchase-time information. The users table includes Google identity/password-login additions from the October 2026 migration.

Source: database/migrations/; docs/DATABASE_REFERENCE.md.

4.5 Data Dictionary (continued 1)

roles

Field(s)	Type, constraints and defaults
id	id()
name	string('name', 40)->unique()
label	string('label', 80)
created_at, updated_at	timestamps()

users

Field(s)	Type, constraints and defaults
id	id()
name	string('name')
email	string('email')->unique()
phone	string('phone', 20)->nullable()
email_verified_at	timestamp('email_verified_at')->nullable()
password	string('password')
role_id	foreignId('role_id')->constrained('roles')->restrictOnDelete()
is_active	boolean('is_active')->default(true)
last_login_at	timestamp('last_login_at')->nullable()
remember_token	rememberToken()
created_at, updated_at	timestamps()
google_id	string('google_id')->nullable()->unique()
password_login_enabled	boolean('password_login_enabled')->default(true)
Indexes / constraints	index(['role_id', 'is_active']).

password_reset_tokens

Field(s)	Type, constraints and defaults
email	string('email')->primary()
token	string('token')
created_at	timestamp('created_at')->nullable()

4.5 Data Dictionary (continued 2)

sessions

Field(s)	Type, constraints and defaults
id	string('id')->primary()
user_id	foreignId('user_id')->nullable()->index()
ip_address	string('ip_address', 45)->nullable()
user_agent	text('user_agent')->nullable()
payload	longText('payload')
last_activity	integer('last_activity')->index()

cache

Field(s)	Type, constraints and defaults
key	string('key')->primary()
value	mediumText('value')
expiration	integer('expiration')

cache_locks

Field(s)	Type, constraints and defaults
key	string('key')->primary()
owner	string('owner')
expiration	integer('expiration')

4.5 Data Dictionary (continued 3)

jobs

Field(s)	Type, constraints and defaults
id	id()
queue	string('queue')->index()
payload	longText('payload')
attempts	unsignedTinyInteger('attempts')
reserved_at	unsignedInteger('reserved_at')->nullable()
available_at	unsignedInteger('available_at')
created_at	unsignedInteger('created_at')

job_batches

Field(s)	Type, constraints and defaults
id	string('id')->primary()
name	string('name')
total_jobs	integer('total_jobs')
pending_jobs	integer('pending_jobs')
failed_jobs	integer('failed_jobs')
failed_job_ids	longText('failed_job_ids')
options	mediumText('options')->nullable()
cancelled_at	integer('cancelled_at')->nullable()
created_at	integer('created_at')
finished_at	integer('finished_at')->nullable()

4.5 Data Dictionary (continued 4)

failed_jobs

Field(s)	Type, constraints and defaults
id	id()
uuid	string('uuid')->unique()
connection	text('connection')
queue	text('queue')
payload	longText('payload')
exception	longText('exception')
failed_at	timestamp('failed_at')->useCurrent()

user_addresses

Field(s)	Type, constraints and defaults
id	id()
user_id	foreignId('user_id')->constrained()->cascadeOnDelete()
label	string('label', 40)->nullable()
name	string('name')
phone	string('phone', 20)
address_line1	string('address_line1')
address_line2	string('address_line2')->nullable()
city	string('city', 100)
state	string('state', 100)
postal_code	string('postal_code', 10)
country	string('country', 60)->default('India')
is_default	boolean('is_default')->default(false)
created_at, updated_at	timestamps()
Indexes / constraints	index(['user_id', 'is_default']).

4.5 Data Dictionary (continued 5)

categories

Field(s)	Type, constraints and defaults
id	id()
name	string('name')
slug	string('slug')->unique()
description	text('description')->nullable()
image_path	string('image_path')->nullable()
sort_order	unsignedInteger('sort_order')->default(0)
is_active	boolean('is_active')->default(true)
created_at, updated_at	timestamps()
Indexes / constraints	index(['is_active', 'sort_order']).

4.5 Data Dictionary (continued 6)

products

Field(s)	Type, constraints and defaults
id	id()
category_id	foreignId('category_id')->nullable()->constrained('categories')->restrictOnDelete()
sku	string('sku', 64)->unique()
name	string('name')
slug	string('slug')->unique()
short_description	string('short_description', 300)->nullable()
description	longText('description')->nullable()
fragrance_family	string('fragrance_family', 60)->nullable()->index()
top_notes	text('top_notes')->nullable()
heart_notes	text('heart_notes')->nullable()
base_notes	text('base_notes')->nullable()
longevity	string('longevity', 120)->nullable()
projection	string('projection', 120)->nullable()
specifications	json('specifications')->nullable()
is_featured	boolean('is_featured')->default(false)
is_bestseller	boolean('is_bestseller')->default(false)
status	string('status', 16)->default('draft')
meta_title	string('meta_title')->nullable()
meta_description	string('meta_description', 320)->nullable()
owner_notes	text('owner_notes')->nullable()
sales_count	unsignedInteger('sales_count')->default(0)
deleted_at	softDeletes()

products (continued)

Field(s)	Type, constraints and defaults
created_at, updated_at	timestamps()
Indexes / constraints	index(['status', 'is_featured']); index(['status', 'is_bestseller']); index(['status', 'created_at']); index('name'); index('sales_count').

4.5 Data Dictionary (continued 7)

product_images

Field(s)	Type, constraints and defaults
id	id()
product_id	foreignId('product_id')->constrained()->cascadeOnDelete()
path	string('path')
alt	string('alt')->nullable()
sort_order	unsignedInteger('sort_order')->default(0)
is_primary	boolean('is_primary')->default(false)
created_at, updated_at	timestamps()
Indexes / constraints	index(['product_id', 'sort_order']).

product_variants

Field(s)	Type, constraints and defaults
id	id()
product_id	foreignId('product_id')->constrained()->cascadeOnDelete()
name	string('name', 80)
size_ml	unsignedSmallInteger('size_ml')->nullable()
sku	string('sku', 64)->unique()
price	decimal('price', 10, 2)->nullable()
sale_price	decimal('sale_price', 10, 2)->nullable()
stock	unsignedInteger('stock')->default(0)
low_stock_threshold	unsignedInteger('low_stock_threshold')->default(5)
is_active	boolean('is_active')->default(true)
sort_order	unsignedInteger('sort_order')->default(0)
created_at, updated_at	timestamps()
Indexes / constraints	index(['product_id', 'is_active']); index('stock').

4.5 Data Dictionary (continued 8)

carts

Field(s)	Type, constraints and defaults
id	id()
user_id	foreignId('user_id')->nullable()->unique()->constrained()->cascadeOnDelete()
session_token	string('session_token', 64)->nullable()->unique()
coupon_code	string('coupon_code', 50)->nullable()
created_at, updated_at	timestamps()

cart_items

Field(s)	Type, constraints and defaults
id	id()
cart_id	foreignId('cart_id')->constrained()->cascadeOnDelete()
product_variant_id	foreignId('product_variant_id')->constrained('product_variants')->cascadeOnDelete()
quantity	unsignedSmallInteger('quantity')->default(1)
created_at, updated_at	timestamps()
Indexes / constraints	unique(['cart_id', 'product_variant_id']).

wishlists

Field(s)	Type, constraints and defaults
id	id()
user_id	foreignId('user_id')->unique()->constrained()->cascadeOnDelete()
created_at, updated_at	timestamps()

4.5 Data Dictionary (continued 9)

wishlist_items

Field(s)	Type, constraints and defaults
id	id()
wishlist_id	foreignId('wishlist_id')->constrained()->cascadeOnDelete()
product_id	foreignId('product_id')->constrained()->cascadeOnDelete()
created_at, updated_at	timestamps()
Indexes / constraints	unique(['wishlist_id', 'product_id']).

coupons

Field(s)	Type, constraints and defaults
id	id()
code	string('code', 50)->unique()
description	string('description')->nullable()
type	string('type', 12)
value	decimal('value', 10, 2)
min_order_amount	decimal('min_order_amount', 10, 2)->nullable()
max_discount	decimal('max_discount', 10, 2)->nullable()
starts_at	timestamp('starts_at')->nullable()
expires_at	timestamp('expires_at')->nullable()
usage_limit	unsignedInteger('usage_limit')->nullable()
per_user_limit	unsignedInteger('per_user_limit')->nullable()
used_count	unsignedInteger('used_count')->default(0)
is_active	boolean('is_active')->default(true)
created_at, updated_at	timestamps()
Indexes / constraints	index(['is_active', 'expires_at']).

coupon_products

Field(s)	Type, constraints and defaults
coupon_id	foreignId('coupon_id')->constrained()->cascadeOnDelete()
product_id	foreignId('product_id')->constrained()->cascadeOnDelete()
Indexes / constraints	primary(['coupon_id', 'product_id']).

4.5 Data Dictionary (continued 10)

coupon_categories

Field(s)	Type, constraints and defaults
coupon_id	foreignId('coupon_id')->constrained()->cascadeOnDelete()
category_id	foreignId('category_id')->constrained()->cascadeOnDelete()
Indexes / constraints	primary(['coupon_id', 'category_id']).

orders

Field(s)	Type, constraints and defaults
id	id()
order_number	string('order_number', 32)->unique()
user_id	foreignId('user_id')->nullable()->constrained()->restrictOnDelete()
idempotency_key	string('idempotency_key', 64)->nullable()->unique()
status	string('status', 24)->default('pending')
payment_method	string('payment_method', 24)
payment_status	string('payment_status', 24)->default('pending')
name	string('name')
email	string('email')
phone	string('phone', 20)
address_line1	string('address_line1')
address_line2	string('address_line2')->nullable()
city	string('city', 100)
state	string('state', 100)
postal_code	string('postal_code', 10)
country	string('country', 60)->default('India')
notes	text('notes')->nullable()
subtotal	decimal('subtotal', 12, 2)
discount_total	decimal('discount_total', 12, 2)->default(0)
shipping_total	decimal('shipping_total', 12, 2)->default(0)
tax_total	decimal('tax_total', 12, 2)->default(0)
grand_total	decimal('grand_total', 12, 2)

4.5 Data Dictionary (continued 11)

orders (continued)

Field(s)	Type, constraints and defaults
refunded_amount	decimal('refunded_amount', 12, 2)->default(0)
coupon_id	foreignId('coupon_id')->nullable()->constrained('coupons')->nullOnDelete()
coupon_code	string('coupon_code', 50)->nullable()
courier_name	string('courier_name', 100)->nullable()
tracking_number	string('tracking_number', 100)->nullable()
tracking_url	string('tracking_url')->nullable()
placed_at	timestamp('placed_at')->nullable()
paid_at	timestamp('paid_at')->nullable()
shipped_at	timestamp('shipped_at')->nullable()
delivered_at	timestamp('delivered_at')->nullable()
cancelled_at	timestamp('cancelled_at')->nullable()
cancel_reason	string('cancel_reason')->nullable()
stock_restored_at	timestamp('stock_restored_at')->nullable()
ip_address	string('ip_address', 45)->nullable()
created_at, updated_at	timestamps()
Indexes / constraints	index(['user_id', 'status']); index('status'); index('payment_status'); index('payment_method'); index('email'); index('created_at').

4.5 Data Dictionary (continued 12)

order_items

Field(s)	Type, constraints and defaults
id	id()
order_id	foreignId('order_id')->constrained()->cascadeOnDelete()
product_id	foreignId('product_id')->nullable()->constrained('products')->restrictOnDelete()
product_variant_id	foreignId('product_variant_id')->nullable()->constrained('product_variants')->restrictOnDelete()
product_name	string('product_name')
variant_name	string('variant_name', 80)->nullable()
sku	string('sku', 64)
description_snapshot	text('description_snapshot')->nullable()
image_path	string('image_path')->nullable()
list_price	decimal('list_price', 10, 2)
unit_price	decimal('unit_price', 10, 2)
quantity	unsignedSmallInteger('quantity')
line_total	decimal('line_total', 12, 2)
created_at, updated_at	timestamps()
Indexes / constraints	index('order_id'); index('product_id').

4.5 Data Dictionary (continued 13)

payments

Field(s)	Type, constraints and defaults
id	id()
order_id	foreignId('order_id')->constrained()->restrictOnDelete()
provider	string('provider', 24)
provider_order_id	string('provider_order_id', 80)->nullable()->index()
provider_payment_id	string('provider_payment_id', 80)->nullable()->unique()
amount	decimal('amount', 12, 2)
currency	string('currency', 3)->default('INR')
status	string('status', 24)->default('created')
method	string('method', 40)->nullable()
error_code	string('error_code', 80)->nullable()
error_description	string('error_description')->nullable()
verified_at	timestamp('verified_at')->nullable()
meta	json('meta')->nullable()
created_at, updated_at	timestamps()
Indexes / constraints	index(['order_id', 'status']).

payment_webhook_events

Field(s)	Type, constraints and defaults
id	id()
provider	string('provider', 24)
event_id	string('event_id', 100)
event_type	string('event_type', 80)
payload	json('payload')->nullable()
processed_at	timestamp('processed_at')->nullable()
outcome	string('outcome')->nullable()
created_at, updated_at	timestamps()
Indexes / constraints	unique(['provider', 'event_id']).

4.5 Data Dictionary (continued 14)

coupon_usages

Field(s)	Type, constraints and defaults
id	id()
coupon_id	foreignId('coupon_id')->constrained()->restrictOnDelete()
order_id	foreignId('order_id')->constrained()->cascadeOnDelete()
user_id	foreignId('user_id')->nullable()->constrained()->nullOnDelete()
email	string('email')
discount_amount	decimal('discount_amount', 10, 2)
created_at, updated_at	timestamps()
Indexes / constraints	unique(['coupon_id', 'order_id']); index(['coupon_id', 'user_id']); index(['coupon_id', 'email']).

order_status_histories

Field(s)	Type, constraints and defaults
id	id()
order_id	foreignId('order_id')->constrained()->cascadeOnDelete()
from_status	string('from_status', 24)->nullable()
to_status	string('to_status', 24)
note	string('note')->nullable()
changed_by	foreignId('changed_by')->nullable()->constrained('users')->nullOnDelete()
created_at, updated_at	timestamps()
Indexes / constraints	index(['order_id', 'created_at']).

4.5 Data Dictionary (continued 15)

inventory_movements

Field(s)	Type, constraints and defaults
id	id()
product_variant_id	foreignId('product_variant_id')->constrained('product_variants')->cascadeOnDelete()
type	string('type', 30)
quantity_change	integer('quantity_change')
stock_after	unsignedInteger('stock_after')
reference_type	string('reference_type', 40)->nullable()
reference_id	unsignedBigInteger('reference_id')->nullable()
user_id	foreignId('user_id')->nullable()->constrained()->nullOnDelete()
note	string('note')->nullable()
created_at	timestamp('created_at')->useCurrent()
Indexes / constraints	index(['product_variant_id', 'created_at']); index(['reference_type', 'reference_id']); index('type').

reviews

Field(s)	Type, constraints and defaults
id	id()
product_id	foreignId('product_id')->constrained()->cascadeOnDelete()
user_id	foreignId('user_id')->constrained()->restrictOnDelete()
order_id	foreignId('order_id')->nullable()->constrained('orders')->nullOnDelete()
rating	unsignedTinyInteger('rating')
title	string('title')->nullable()
body	text('body')
status	string('status', 16)->default('pending')
moderated_by	foreignId('moderated_by')->nullable()->constrained('users')->nullOnDelete()
moderated_at	timestamp('moderated_at')->nullable()
created_at, updated_at	timestamps()
Indexes / constraints	unique(['product_id', 'user_id']); index(['product_id', 'status']); index('status').

4.5 Data Dictionary (continued 16)

contact_enquiries

Field(s)	Type, constraints and defaults
id	id()
name	string('name')
email	string('email')
phone	string('phone', 20)->nullable()
subject	string('subject')->nullable()
message	text('message')
status	string('status', 20)->default('new')
admin_notes	text('admin_notes')->nullable()
ip_address	string('ip_address', 45)->nullable()
handled_by	foreignId('handled_by')->nullable()->constrained('users')->nullOnDelete()
created_at, updated_at	timestamps()
Indexes / constraints	index('status'); index('created_at').

newsletter_subscribers

Field(s)	Type, constraints and defaults
id	id()
email	string('email')->unique()
consent_given_at	timestamp('consent_given_at')
consent_ip	string('consent_ip', 45)->nullable()
unsubscribe_token	string('unsubscribe_token', 64)->unique()
unsubscribed_at	timestamp('unsubscribed_at')->nullable()
created_at, updated_at	timestamps()

4.5 Data Dictionary (continued 17)

banners

Field(s)	Type, constraints and defaults
id	id()
title	string('title')
subtitle	string('subtitle')->nullable()
image_path	string('image_path')->nullable()
button_text	string('button_text', 60)->nullable()
link_url	string('link_url')->nullable()
position	string('position', 20)->default('promo')
sort_order	unsignedInteger('sort_order')->default(0)
is_active	boolean('is_active')->default(true)
starts_at	timestamp('starts_at')->nullable()
ends_at	timestamp('ends_at')->nullable()
created_at, updated_at	timestamps()
Indexes / constraints	index(['position', 'is_active', 'sort_order']).

pages

Field(s)	Type, constraints and defaults
id	id()
slug	string('slug')->unique()
title	string('title')
content	longText('content')->nullable()
meta_title	string('meta_title')->nullable()
meta_description	string('meta_description', 320)->nullable()
is_published	boolean('is_published')->default(false)
reviewed_at	timestamp('reviewed_at')->nullable()
reviewed_by	foreignId('reviewed_by')->nullable()->constrained('users')->nullOnDelete()
created_at, updated_at	timestamps()

4.5 Data Dictionary (continued 18)

faqs

Field(s)	Type, constraints and defaults
id	id()
question	string('question')
answer	text('answer')
sort_order	unsignedInteger('sort_order')->default(0)
is_active	boolean('is_active')->default(true)
created_at, updated_at	timestamps()
Indexes / constraints	index(['is_active', 'sort_order']).

settings

Field(s)	Type, constraints and defaults
id	id()
key	string('key')->unique()
value	text('value')->nullable()
group	string('group', 40)->default('general')
type	string('type', 16)->default('string')
created_at, updated_at	timestamps()
Indexes / constraints	index('group').

4.5 Data Dictionary (continued 19)

admin_activity_logs

Field(s)	Type, constraints and defaults
id	id()
user_id	foreignId('user_id')->nullable()->constrained()->nullOnDelete()
action	string('action', 60)
subject_type	string('subject_type', 60)->nullable()
subject_id	unsignedBigInteger('subject_id')->nullable()
description	string('description')
properties	json('properties')->nullable()
ip_address	string('ip_address', 45)->nullable()
user_agent	string('user_agent')->nullable()
created_at	timestamp('created_at')->useCurrent()
Indexes / constraints	index('action'); index(['subject_type', 'subject_id']); index('created_at').

5. DEVELOPMENT

5.1 Development Approach

The implementation uses a layered, server-rendered Laravel architecture. Controllers coordinate requests, Form Requests validate structured input, services implement business operations, Eloquent models represent persistence, and Blade views render the interface. Browser libraries add local interactions, motion, QR generation and dashboard charts.

The repository demonstrates modular implementation and automated checks. The available evidence does not establish a historical sprint cadence, daily meetings or individual commit contributions. The staged process below is a practical description of how the modules are organized and should be maintained.

1. Identify users, product information, payment methods and order rules.
2. Design the database and relationships through versioned migrations.
3. Implement public discovery, authentication and customer account screens.
4. Implement shared cart, pricing, coupon, order and inventory services.
5. Add verified payment flows, administration, content and reports.
6. Exercise isolated tests and preview screens; record gaps before release.

Important implementation decisions

- Prices and stock belong to product variants rather than only the parent product.
- Checkout runs within a transaction, locks variants in stable order and revalidates current price/stock.
- Order placement reserves stock immediately and records purchase-time snapshots.
- Payment and fulfilment states are independent.
- Runtime business settings use the settings table; secrets remain in server environment configuration.

Source: app/Http/; app/Services/; app/Models/; resources/views/.

5.2 Coding Standards

Element	Convention used in this Laravel project
Classes	PascalCase: OrderService, ProductVariant, CheckoutController.
Methods / variables	camelCase: calculate(), verifyReceipt(), orderNumber.
Constants	UPPER_SNAKE_CASE: STATUS_ACTIVE, REFERENCE_RULE.
Database tables / columns	Plural snake_case tables; snake_case columns and foreign keys.
Routes	Named routes; public slugs and human order numbers where configured.
Indentation	Consistent PHP indentation, generally four spaces.
Templates	Blade layouts/components with escaped output and CSRF-protected forms.
Business logic	Shared services for order, pricing, inventory and payment rules.

Validation and error handling

Use CheckoutRequest and ProductRequest for structured validation, with controller/service checks for method availability, ownership and state transitions. Keep failures meaningful to the user without exposing server secrets. The application includes dedicated 403, 404, 419, 429, 500 and 503 error views.

Authentication and sensitive data

Passwords use Laravel hashing rather than plain text or a generic SHA-256 password scheme. Google OAuth linking requires the explicit account flow; matching an existing email alone does not silently link accounts. Uploaded raster images are type/size/dimension validated and stored under random names.

Money and transactions

Use Money conversions and PricingService instead of duplicating floating-point total calculations in templates or controllers. Apply inventory mutations through InventoryService and retain order status and stock movement history.

Source: app/Support/Money.php; app/Http/Requests/; app/Services/; config/hashing.php.

5.3 GitHub Repository Details

Item	Verified project information
Local project directory	C:\xampp\htdocs\arsh-perfume
Application package	arshperfume/store
Source control	A local .git directory exists.
GitHub repository URL	No configured remote was found during inspection; a public/private repository URL is not supplied.
Contribution history	No verified commit-based contribution history is asserted in this report.
Dependency versions	composer.lock and package-lock.json are present for reproducible installation.

Recommended repository organization

```
app/                Controllers, services, models and policies
bootstrap/          Application boot and middleware
config/             Framework and business configuration
database/           Migrations, factories and seeders
resources/          Blade templates, CSS and JavaScript
routes/             Public, auth, admin and scheduled routes
tests/              Unit and feature tests
docs/               Project documentation and generators
public/             Web document root and built assets
```

Version-control handover

When a repository is configured, keep reviewable commits, document the branching approach actually chosen and exclude .env, credentials, runtime logs and private customer data. Do not insert an invented GitHub URL or fabricated commit history into an academic submission.

Source: composer.json; .gitignore; composer.lock; package-lock.json.

5.4 AI Tools / Libraries / APIs Used

Tool / dependency	Version family	Purpose in this project
PHP / Laravel	PHP ^8.2 / Laravel ^12.0	Application runtime, routing, validation, sessions, ORM, queues and notifications.
Laravel Socialite	^5.31	Google OAuth identity redirect/callback and profile mapping.
Tailwind CSS / Vite	4 / 7	CSS build pipeline, browser modules and production assets.
Alpine.js	^3.14.9	Lightweight browser interactions in server-rendered screens.
Chart.js	^4.4.9	Administrator charts and report visualization.
Framer Motion	^14.6.0	DOM-based storefront motion without a React SPA.
qrcode	^1.5.4	Generate UPI QR codes locally in the browser.
Axios	^1.11.0	HTTP utility available to browser modules.
Razorpay API	Configured provider API	Payment order/capture verification and eligible live refunds.
Google OAuth	Configured web OAuth client	Optional customer/admin account sign-in and explicit linking.
PHPUnit	^11.5	Unit and feature verification with in-memory SQLite.

AI usage disclosure

The application source does not contain an integrated AI prediction, chatbot or LLM API feature. Codex assisted with source inspection and preparation of this documentation. No unsupported claim about the original team's use of other AI tools is made.

> Versions above are the dependency constraints declared in this project, not a claim that they are the newest available releases.

Source: composer.json; package.json; app/Services/RazorpayService.php; config/services.php.

5.5 Screenshots & Implementation Details

Screenshot 1: Storefront / Home

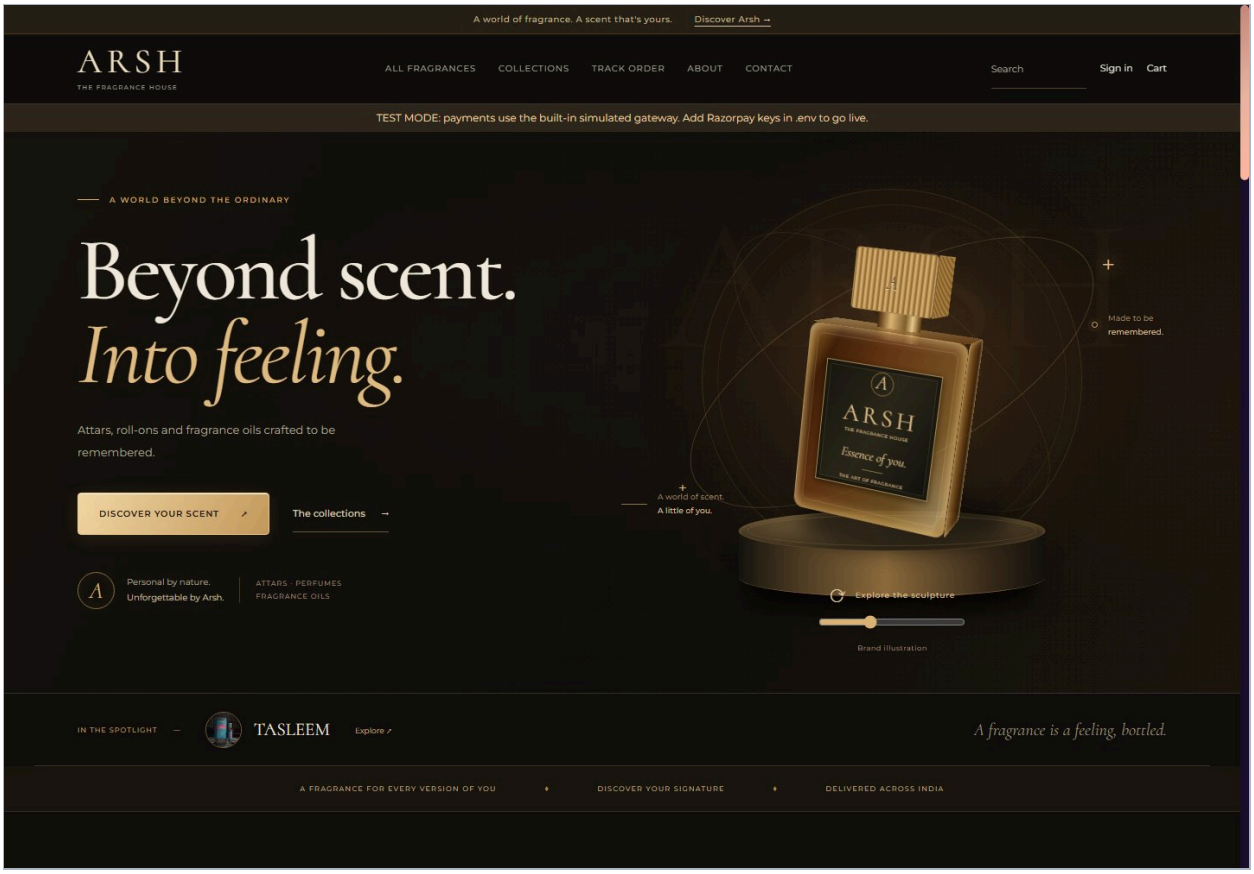


Figure 5.1: Saved desktop preview of the ARSH PERFUME home screen.

The homepage presents the fragrance-house identity, primary navigation and collection discovery. The visual product sculpture is a brand illustration; actual catalogue photography is maintained separately.

- Blade layouts render navigation, page sections and store settings.
- Storefront CSS and browser motion enhance the branded presentation.
- The saved capture displays the local simulated-payment notice.

> Screenshots in this section are existing project preview captures. They document the interface, not current live stock, real sales totals or successful provider transactions.

5.5 Screenshots (continued)

Screenshot 2: Catalogue and Search

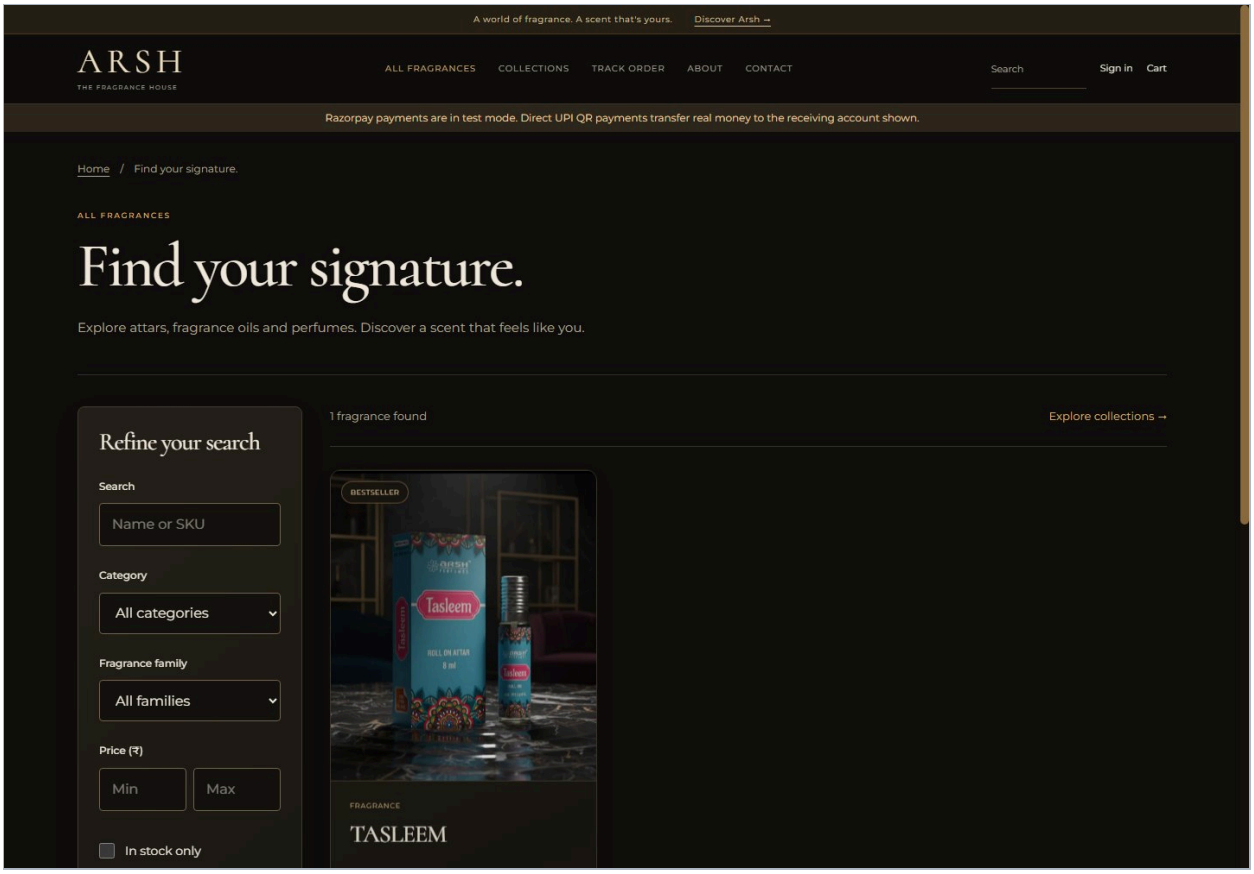


Figure 5.2: Saved desktop catalogue preview with search/filter controls.

The catalogue allows keyword search and filtering by category, fragrance family, price and stock availability. Product cards show available catalogue content; variant pricing and stock determine whether a size can be purchased.

- ShopController validates allowed query filters and sort values.
- Eloquent catalogue scopes load visible products and variants.
- The cart and checkout revalidate current stock and price independently of the displayed card.

Source: app/Http/Controllers/ShopController.php; resources/views/pages/shop.blade.php.

5.5 Screenshots (continued)

Screenshot 3: Administrator Dashboard

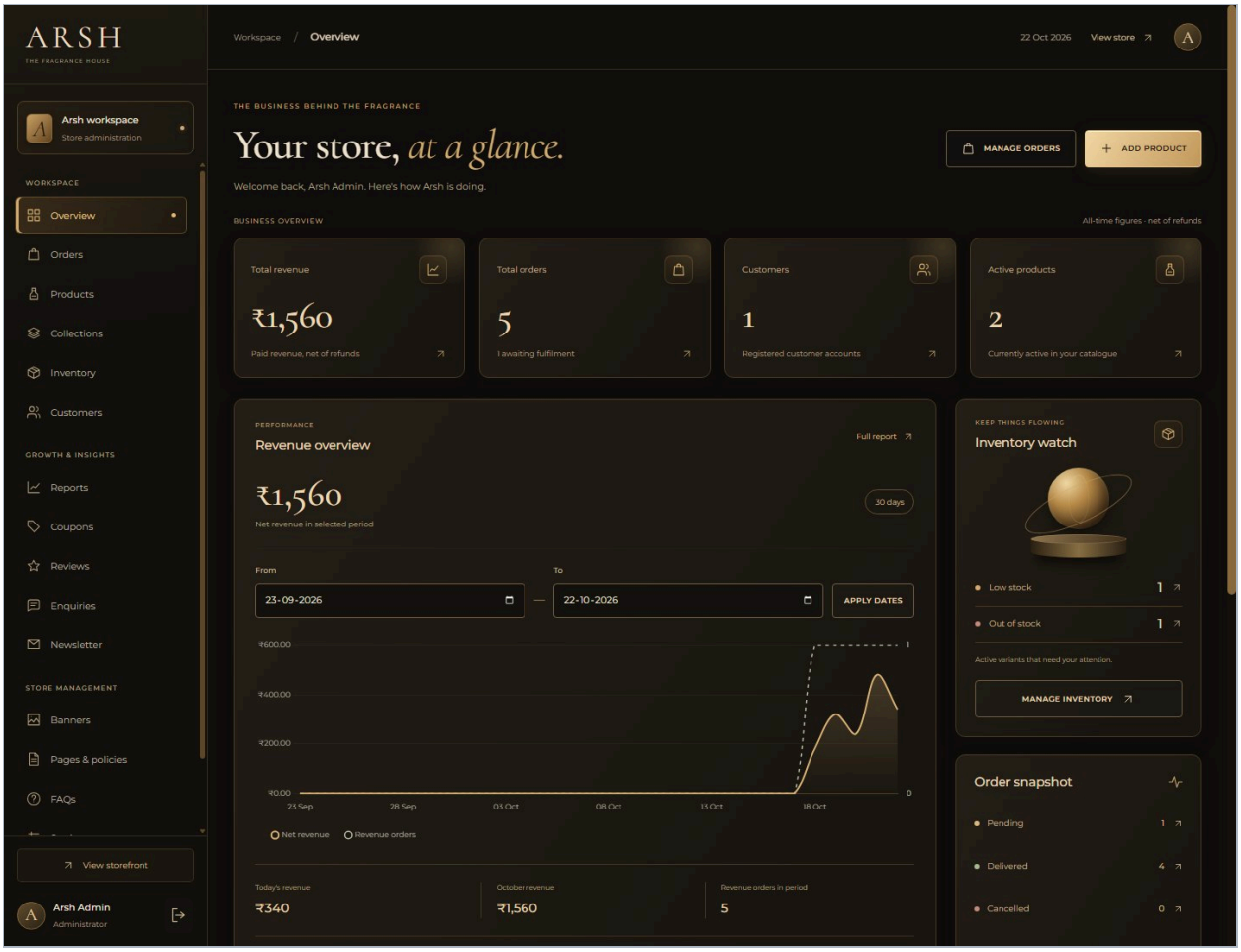


Figure 5.3: Saved administrator workspace preview; displayed figures are capture-time values.

The administrator dashboard groups order/revenue information, catalogue counts, charting and stock attention indicators. Navigation opens order, product, inventory, customer, report and content modules.

- The admin route group requires authentication and an active administrator.
- ReportService supplies shared metrics; Chart.js renders charts.
- Revenue excludes cancelled/returned orders and nets recorded refunds.

Source: app/Http/Controllers/Admin/DashboardController.php; app/Services/ReportService.php.

5.5 Implementation Details (continued)

Authentication and customer access

Email registration/login, password reset and optional Google sign-in are implemented through dedicated controllers. My Account supports profile, password and address changes. Ownership policies protect customer order and address actions. Disabled users are signed out by active-account middleware.

Checkout, payments and stock

OrderService rechecks stock and prices, locks variants in stable ID order, validates coupons, saves purchase snapshots and reserves stock in a transaction. COD collection is recorded separately. Razorpay uses server totals, signed callbacks/webhooks and duplicate-event handling. UPI references remain unverified until an administrator checks actual receipt.

Fulfilment and content

The configured order lifecycle is pending → confirmed → processing → shipped → out for delivery → delivered, with permitted cancellation and return branches. Administrators can add tracking information, view invoices and record refunds. Pages, FAQs, banners and homepage settings provide content management.

Implementation rule	Result
Customer cancellation	Allowed for own pending/confirmed orders only.
Stock restoration	Cancellation restores reserved stock once.
Razorpay timeout	Unpaid reservations older than 30 minutes are released by the scheduled task.
UPI timeout	Direct UPI awaiting review is excluded; review/cancel manually.
UPI refund	Bank transfer happens outside the store before the refund record is entered.

Source: config/arsh.php; app/Services/OrderService.php; app/Services/UpiPaymentService.php.

5.6 Installation and Configuration

This additional section supplies the setup information needed to reproduce the submitted project. The commands describe a fresh local installation; preserve an existing environment file, application key and owner data when working with an installed store.

```
Set-Location C:\xampp\htdocs\arsh-perfume
composer install
# Only if .env does not already exist:
Copy-Item .env.example .env
php artisan key:generate
# Create the MySQL database and set DB_* in .env.
php artisan migrate --seed
npm ci
npm run build
php artisan storage:link
php artisan arsh:create-admin owner@example.com --name="Store Owner"
php artisan serve
```

The administrator command prompts for a password. Use the address actually opened in the browser as APP_URL. For Apache/XAMPP, point the virtual-host document root at public/ and enable rewriting. Ensure storage/ and bootstrap/cache/ are writable by the web process.

Configuration groups

Configuration	Where it belongs
Database, application key, mail and payment secrets	Server .env / deployment environment; never in frontend code.
Google OAuth client and callback	Server configuration; callback must match scheme, hostname, port and path.
Business identity, shipping/tax, COD, guest checkout and UPI payee	Admin → Settings; reviewed by the store owner.
Initial product/policy content	Draft/unpublished seeds; owner confirms content before publication.

Source: .env.example; composer.json; config/arsh.php; docs/google-login-setup.md.

5.7 Testing and Validation

The documented isolated test run on 27-10-2026 used the testing environment and in-memory SQLite. The existing automated suite returned **68 passed, 2 failed and 389 assertions**. This is the observed result, not a claim that every deployment or live integration has passed.

Test area	Representative coverage
Money / pricing	Paise conversion, monetary formatting and checkout totals.
Access control	Admin/customer separation, ownership and guest access.
Catalogue / stock	Publishing gates, sale price rules, stock adjustments and movement records.
Checkout / orders	Availability, coupon validation, price snapshots, status transitions and one-time stock restoration.
Payments	Forged signatures, captures, duplicate events, amount mismatch and UPI receipt verification.
Google / rendering	Account linking, role protection, OAuth state and storefront/admin rendering.

Existing failures recorded

1. AdminProductTest image-storage test: CLI PHP lacks GD, so the fixture image cannot be generated.
2. PaymentWebhookTest wrong-amount test: the payment-attempt assertion expects pending, while the implementation retains created. The order itself remains pending.

Checks still required before production

Resolve the two failures and rerun the suite. Exercise the real configured Razorpay/Google paths, mail delivery, scheduler/worker operation, browser flows and backup restoration. A real bank transfer, live gateway charge/refund and production-load benchmark are not claimed here.

```
php artisan test --compact
php artisan route:list
npm run build
```

Source: phpunit.xml; tests/Unit/; tests/Feature/.

5.8 Deployment and Maintenance

1. Deploy PHP/MySQL with HTTPS and public/ as the web document root.
2. Set production environment, disable debug output and use secure session cookies.
3. Install locked dependencies, build assets, back up data and apply versioned migrations.
4. Verify uploaded-media access and cache application configuration/routes/views.
5. Run a supervised queue worker and invoke the scheduler every minute.
6. Confirm products, policies, taxes, shipping, notifications and payment methods before opening the store.

```
composer install --no-dev --optimize-autoloader
npm ci
npm run build
php artisan migrate --force
php artisan config:cache
php artisan route:cache
php artisan view:cache
php artisan queue:restart
# Persistent worker:
php artisan queue:work --sleep=3 --tries=3 --max-time=3600
# Run once per minute through the server scheduler:
php artisan schedule:run
```

Operational responsibilities

- Inspect application logs and failed jobs; maintain backups of the database and uploaded media.
- Verify restore procedures and retain protected deployment configuration.
- Review stale UPI orders manually and verify receipts before confirmation.
- Check scheduled unpaid Razorpay expiry and daily failed-job pruning.
- Update the report when schema, interfaces, integrations or validation results change.

Source: routes/console.php; bootstrap/app.php; config/queue.php.

6. PROPOSED ENHANCEMENTS

The following items are future work. They are not presented as existing features of the submitted application.

Priority	Enhancement	Expected benefit / dependency
Immediate	Resolve current test failures	Enable CLI GD and reconcile the expected/implemented payment-attempt state.
Immediate	Complete CSV product import	Implement validation, row-level feedback and safe catalogue updates for the existing route.
High	Payment reconciliation tools	Help administrators compare pending UPI orders, receipt references and recorded refunds.
High	Image optimization and delivery	Generate appropriately sized images and evaluate caching/CDN support.
Medium	Drag-and-drop category ordering	Improve administrative catalogue organization.
Medium	Transactional SMS / WhatsApp	Add optional order updates through a configured provider and consent workflow.
Medium	Advanced search	Evaluate a dedicated search engine when catalogue size requires it.
Later	Additional languages / currencies	Extend content and pricing only after suitable business and integration rules are designed.
Later	Mobile application / PWA	Provide an installable experience with defined account, checkout and notification behavior.
Later	Product recommendations	Evaluate consent, data quality and measurable usefulness before selecting rules or AI methods.

Prioritize correctness, reproducible tests, live integration verification and recoverability before expanding the product scope.

7. CONCLUSION

ARSH PERFUME brings fragrance discovery, shopping, order processing and store administration into one web application. The implementation includes a responsive storefront, product variants, carts, coupons, customer accounts, multiple payment paths, inventory tracking, fulfilment tools and owner-managed content.

The principal engineering contribution is the connection between order integrity and day-to-day retail operations: totals are calculated on the server, stock is reserved transactionally, purchase details are snapshotted, ownership is checked and payment confirmation is kept separate from customer-entered claims. The administration panel adds the operational tools required to maintain this information.

Technical learning and challenges represented by the project

- Separating request handling, reusable business rules and persistence.
- Handling simultaneous stock changes and repeated order/payment events.
- Protecting customer data while supporting both guests and registered buyers.
- Integrating external identity/payment services without trusting browser-only outcomes.
- Building responsive customer and administration views with a shared backend.
- Using tests and documentation to expose unresolved environment or behavior gaps.

Current submission status

The source, diagrams, schema dictionary, screenshots and setup/operation instructions are documented in the format of the supplied UDP reference. The known two test failures and incomplete CSV import remain explicit. Real configured provider flows and deployment operation require separate verification before a production-readiness claim can be made.

The project provides a maintainable foundation for an online fragrance store. Future work should first strengthen verification and reconciliation, then add capabilities that are justified by customer and store-owner needs.

8. BIBLIOGRAPHY

[1] Laravel, *Laravel 12.x Documentation*. https://laravel.com/docs/12.x. Accessed 27-10-2026. Framework architecture, routing, validation, ORM, queues and application setup.

[2] Laravel, *Laravel Socialite — Laravel 12.x*. https://laravel.com/docs/12.x/socialite. Accessed 27-10-2026. Google OAuth integration framework.

[3] PHP Documentation Group, *PHP Manual*. https://www.php.net/manual/en/. Accessed 27-10-2026. PHP language, runtime and extension reference.

[4] Oracle / MySQL, *MySQL 8.0 Reference Manual*. https://dev.mysql.com/doc/refman/8.0/en/. Accessed 27-10-2026. Relational database and InnoDB reference.

[5] Tailwind Labs, *Installing Tailwind CSS with Vite*. https://tailwindcss.com/docs/installation/using-vite. Accessed 27-10-2026. Tailwind and Vite asset integration.

[6] Vite, *Getting Started*. https://vite.dev/guide/. Accessed 27-10-2026. Frontend build and development workflow.

[7] Razorpay, *PHP Payment Integration Steps*. https://razorpay.com/docs/payments/server-integration/php/integration-steps/. Accessed 27-10-2026. Checkout/payment integration and verification guidance.

Project primary sources

[8] ARSH PERFUME source repository: composer.json, package.json, routes/, app/, database/migrations/, resources/ and tests/. Inspected 27-10-2026. These define the application-specific behavior described in this report.

[9] Existing project references: docs/ARCHITECTURE.md, docs/DEVELOPER_GUIDE.md, docs/DATABASE_REFERENCE.md, docs/CONFIG_REFERENCE.md and docs/google-login-setup.md. Current source takes precedence over outdated historical notes.

[10] Supplied academic format: udpdocumentation.docx, provided in the local Downloads directory. Used for the chapter order, subsection structure and submission-page format; the unrelated sample-project content was replaced.

> No personal credentials, private database rows or real bank details are included. Saved interface captures are identified as previews. The report date is the supplied submission date, 28-10-2026.